use a hash table to store link to children), plus a variable amount depending on the number of actual children.

- Each link in a tree will require 8 bytes (64 bits references), and each link in a trie will require 9 bytes (8 for the reference plus 1 for the character to which it's associated).
- Each node in the BST will require as many bytes as the number of characters in its key, plus 4 bytes[11] for the Node object itself.
- Each node in the trie requires 1 bit (to hold the Boolean) plus the same constant amount as for the BST: let's round up to 5 bytes.

Given these premises, we note that the BST in figure 6.1 has 9 nodes (whose keys are strings) and consequently `2*9=18` links, while the trie in figure 6.3, has 19 nodes and 18 links. For the BST, the root node contains the key "`anthem`" and requires 27 bytes (4 for the Node itself, 7 for the string, 2*8 for the links); likewise, its left child, with key "`an`", requires 23 bytes – you get how it's computed, it's 21 bytes per node, plus the length of the string. For the whole tree, considering it has 9 nodes and requires a total of 47 bytes for the keys, we need 227 bytes.

Let's now check the trie: each node requires 5 bytes, and each link 9 bytes: total 257 bytes.

So, in practice, this trie might require a little more memory than the corresponding BST. These quantities depend on many factors: first of all, the overhead for objects: since this